

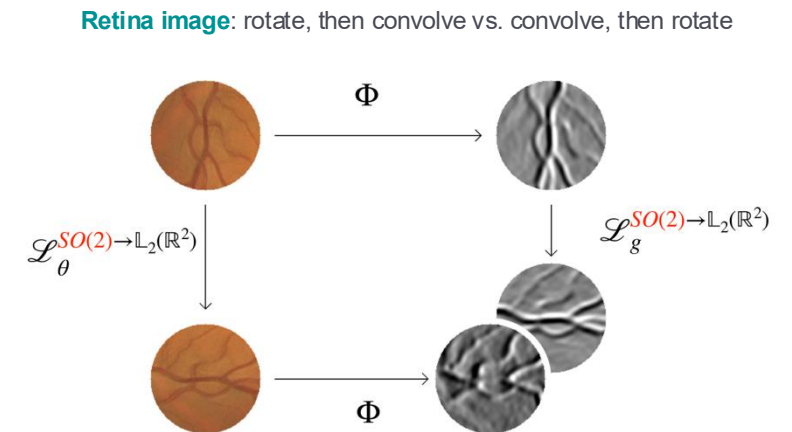
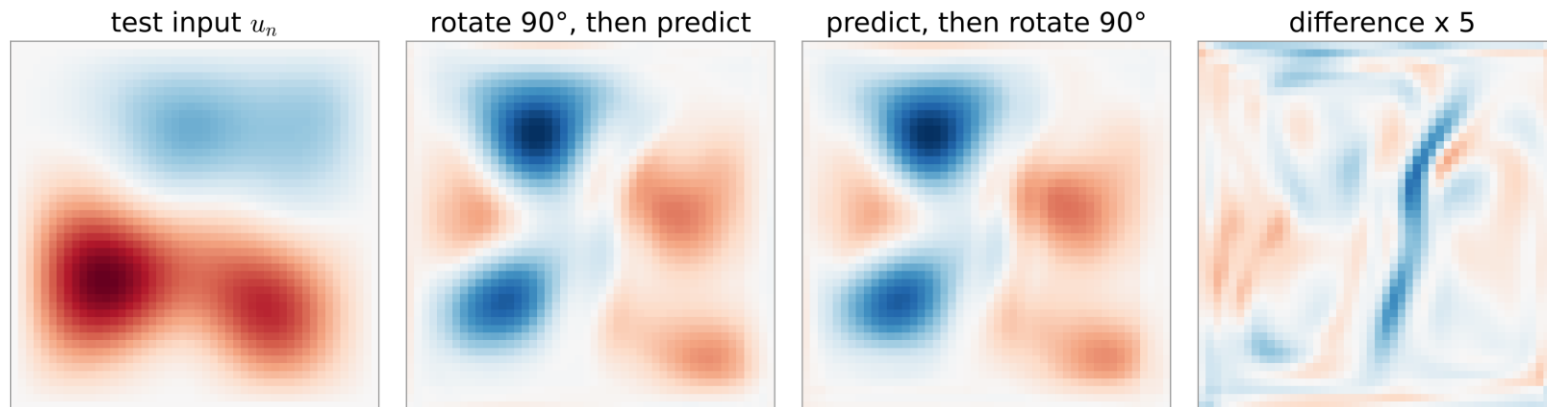


ML FOR SCIENCE · LECTURE 5

Symmetry, Invariance, and Equivariance

Does a CNN commute with rotations?

- Take the SmallCNN from Lecture 4 (heat data, standardized increment). Predict from a test frame, and from the same frame rotated by 90°
- The heat equation on the unit square has no preferred direction, so the two predictions should be rotations of each other
- The predictions differ for every rotation and reflection, by **a sizable fraction of the predicted change**
- CNNs learn rotation symmetry only approximately, from data



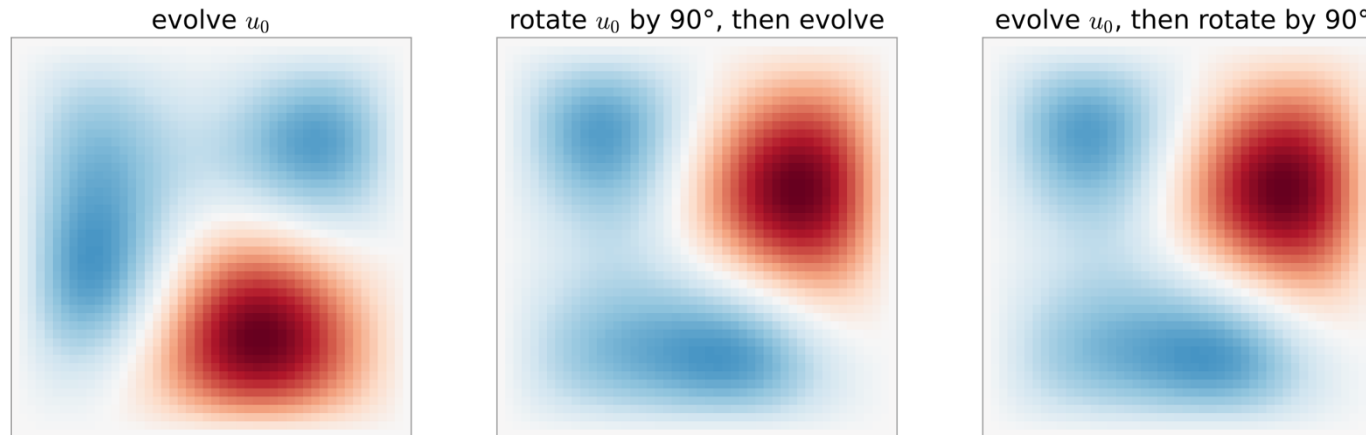
A convolution commutes with translations but not with rotations

A symmetry maps solutions to solutions

- A transformation **g** (rotation, reflection, translation) is a **symmetry** of a PDE problem if it maps every solution to another solution
- Equivalently, transforming the initial condition and then solving gives the same result as solving and then transforming:

$$\mathcal{G}(g \cdot u_0) = g \cdot \mathcal{G}(u_0) \quad \bullet \quad \mathcal{G}: \text{the solution operator; } g \cdot u_0: \text{the transformed initial condition}$$

heat equation, $u = 0$ on the walls, $t = 0.012$ (color scaled at each time)



last two panels agree to round-off (max difference $6e-16$)

If **g** is a symmetry of the physics, a model of **G** should also commute with **g**.

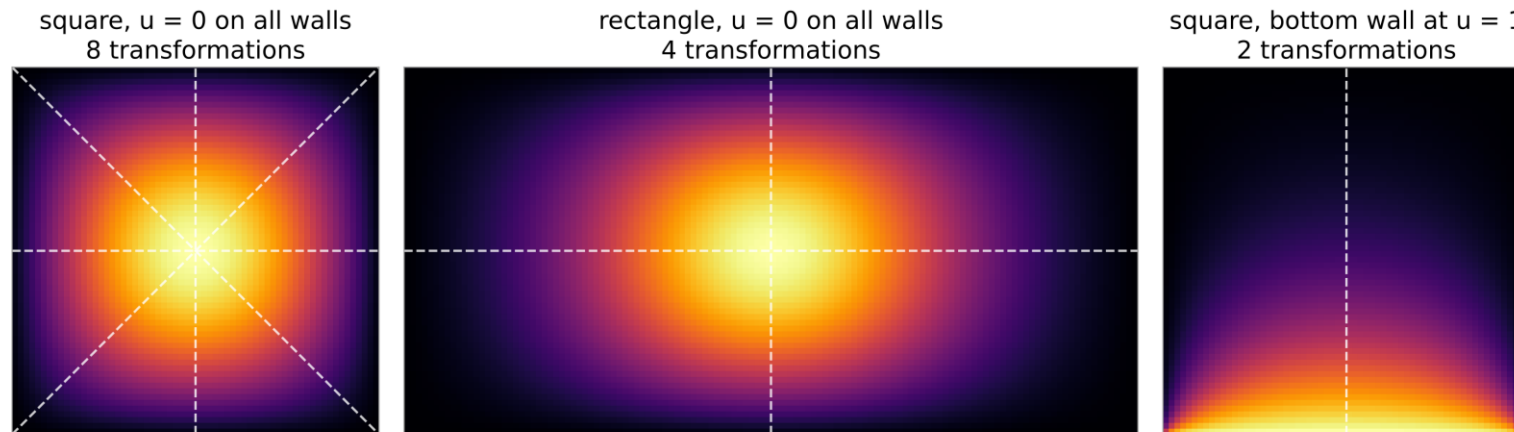
The heat equation has many symmetries

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u, \quad \nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$$

- Each symmetry comes from a property of the equation:
 - **translations (shifts in space)**: the diffusivity α is the same everywhere
 - **shifts in time**: α is constant in time
 - **rotations and reflections**: ∇^2 weighs all directions equally (isotropic)
 - **scaling** $u \rightarrow cu$ and **superposition**: the equation is linear
- Burgers' equation is symmetric under translation, reflection and rotation, but **not under scaling** $u \rightarrow cu$ since it is nonlinear
- Navier-Stokes adds **Galilean invariance**: the same flow seen from a frame moving at constant velocity is also a solution
- Noether (1918): in mechanics, every continuous symmetry gives a conserved quantity. Translation in time gives energy, translation in space gives momentum, rotation gives angular momentum

Symmetries depend on the domain and boundary conditions

- A symmetry must leave the domain and boundary conditions unchanged
 - **square**, $u = 0$ on all walls: rotations by 0° , 90° , 180° , 270° and 4 reflections, **8 symmetries**
 - **rectangle**: rotation by 180° , 2 reflections and the identity, **4 symmetries**
 - **square with a heated bottom wall**: left-right reflection and the identity, **2 symmetries**
- Starting from a hot spot with all 8 symmetries, each solution keeps only the symmetries of its problem

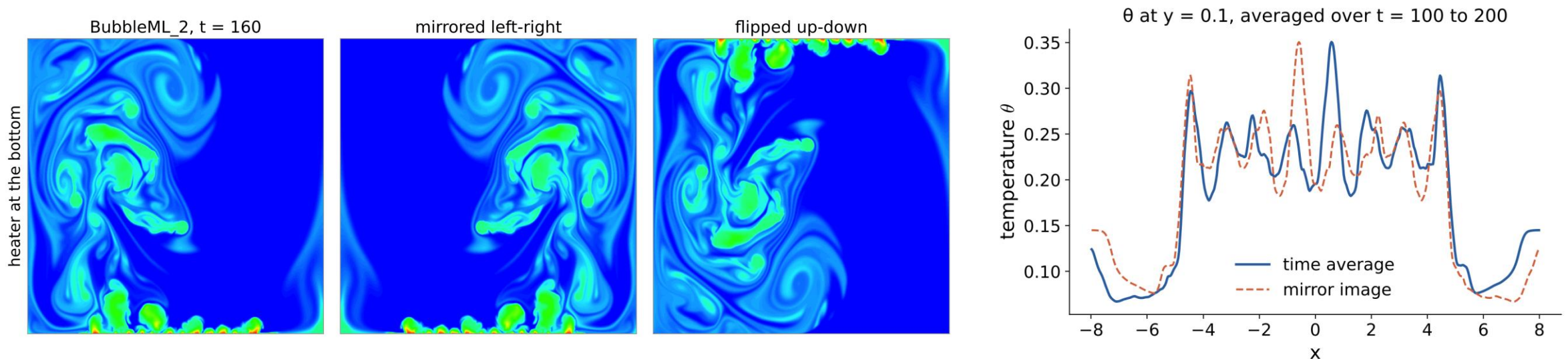


same equation, same centered hot spot at step 0; color scaled to each panel's maximum; step 1500

Symmetry depends on the entire problem: **equation**, **domain**, and **boundary conditions**.

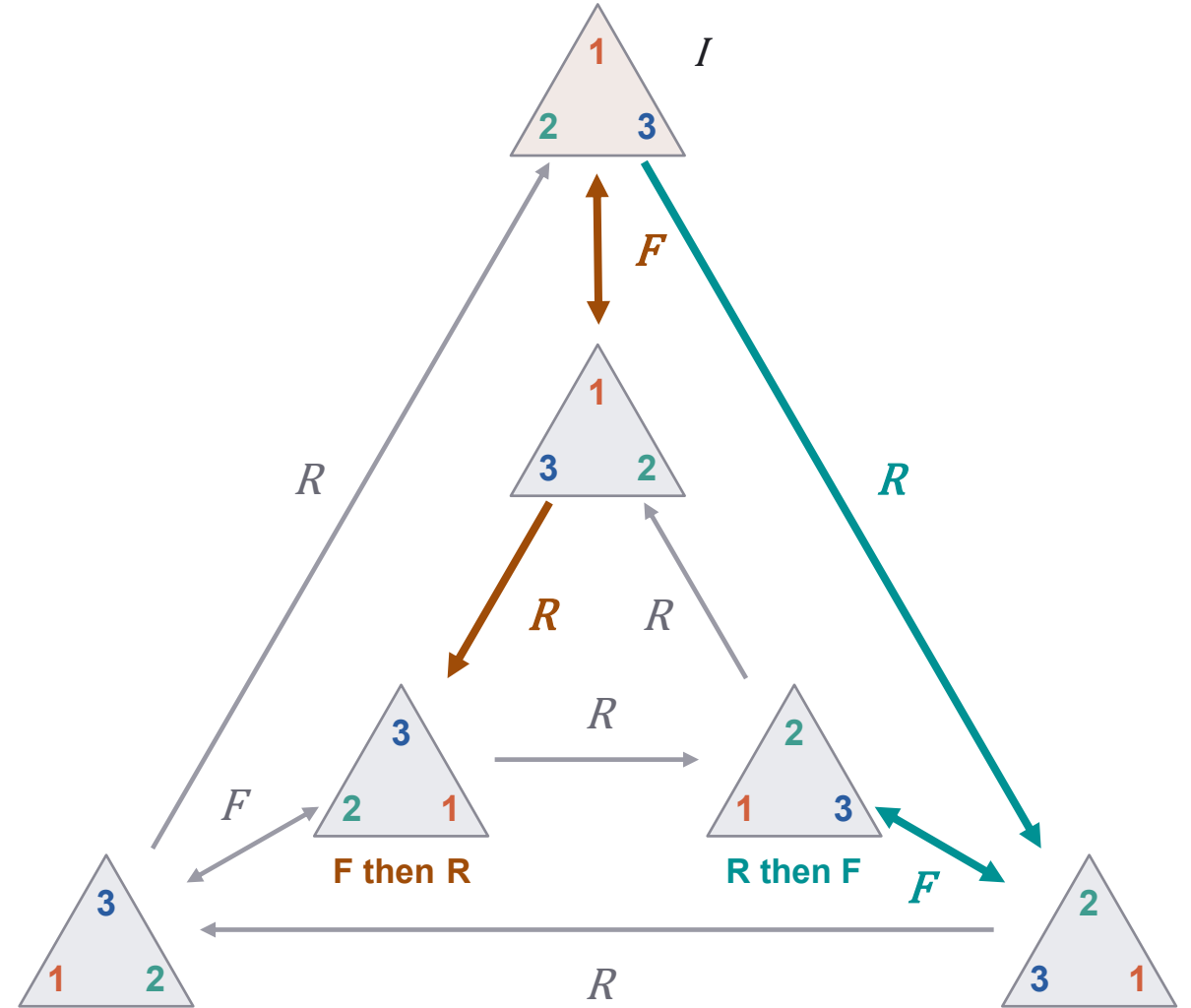
Gravity breaks the symmetry of pool boiling

- BubbleML has the heater at the bottom and gravity pointing down. So, **rotations and up-down flips** break the physics.
- A **left-right mirror** image is a valid solution: the same flow over a heater with mirrored nucleation sites.
- A single frame is not symmetric. A symmetry maps one solution to another; it does not require each solution to be symmetric.
- The peaks on the right are nucleation sites. They are at asymmetric positions, so the **time average is asymmetric**.



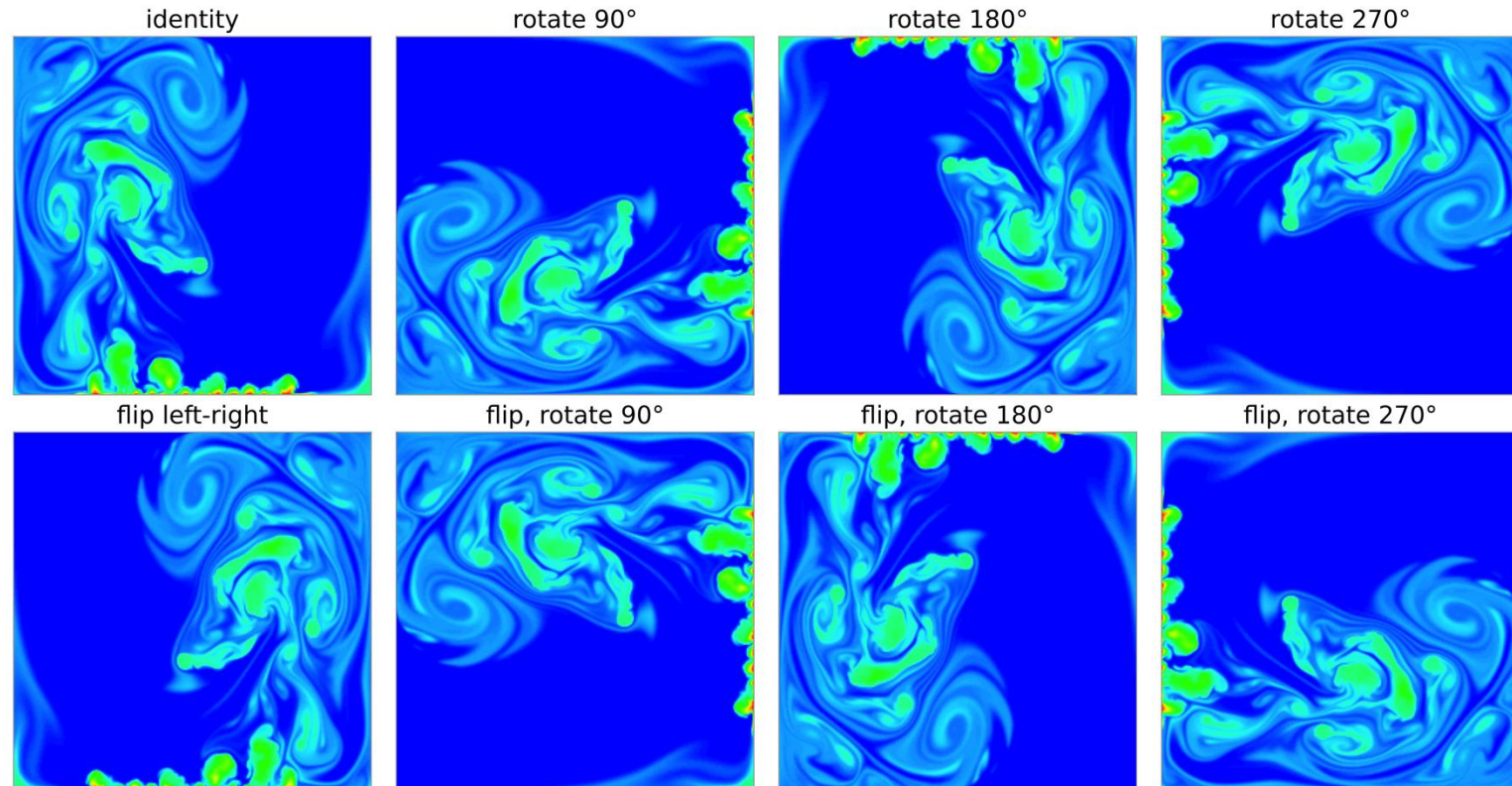
Symmetries form a group

- An equilateral triangle has **6 symmetries**: rotations R by 0° , 120° and 240° , and 3 reflections F
- Two symmetries applied one after the other give another symmetry (**composition**)
- Doing nothing is a symmetry (**identity** I), and every symmetry can be undone (the **inverse**)
- A set of transformations with these properties is a **group**
- Order matters: **R then F** and **F then R** result in different triangles



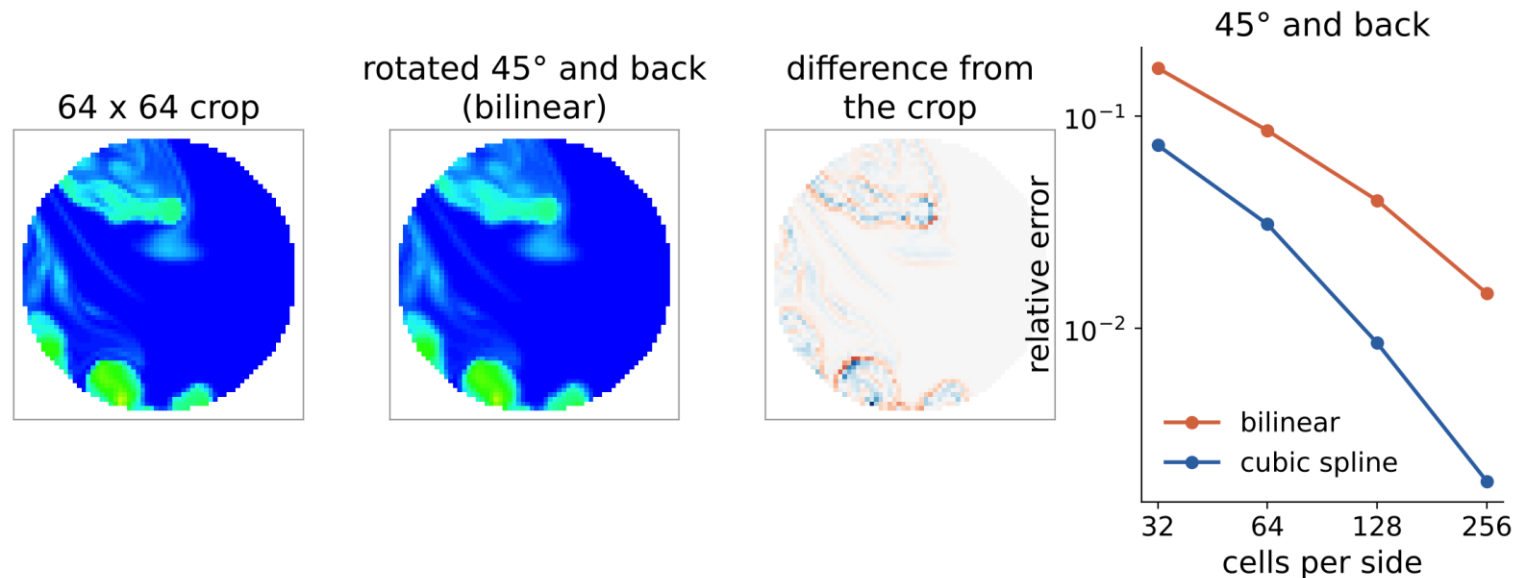
The symmetries of the square form the group D4

- The square has **8 symmetries**: 4 rotations, and 4 reflections, each a left-right flip followed by a rotation
- They form the **dihedral group D4**, which acts the same way on any square array
- For the heat problem on the square, all 8 are symmetries
- For BubbleML, only the identity and the left-right flip are symmetries. The other transformations place the heater on a side or at the top



Discrete and continuous groups

- **Discrete** groups have finitely many elements: D4 (8), all orderings of N points in a point cloud (N!)
- **Continuous** groups have infinitely many: rotations by any angle, translations by any distance, rotations and translations together; in 3D, the rotations, reflections and translations of a molecule
- On a grid, D4 and translations by entire cells reorder the array: 4 rotations by 90° reproduce the original array exactly
- Any other angle moves values off the grid points and needs **interpolation**, which smooths sharp features



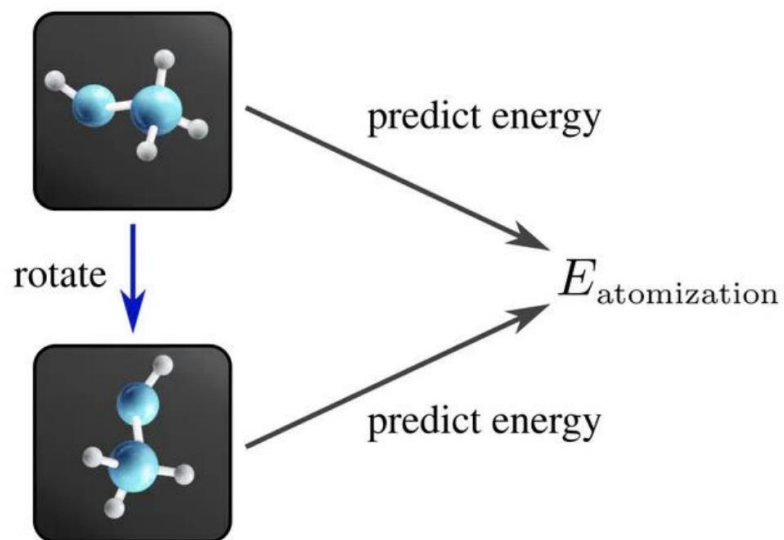
Rotating by an arbitrary angle adds interpolation error and the error grows as the grid gets coarser.

Invariance and equivariance

- A model **f** is **invariant** to **g** if transforming the input leaves the output unchanged:

$$f(g \cdot x) = f(x)$$

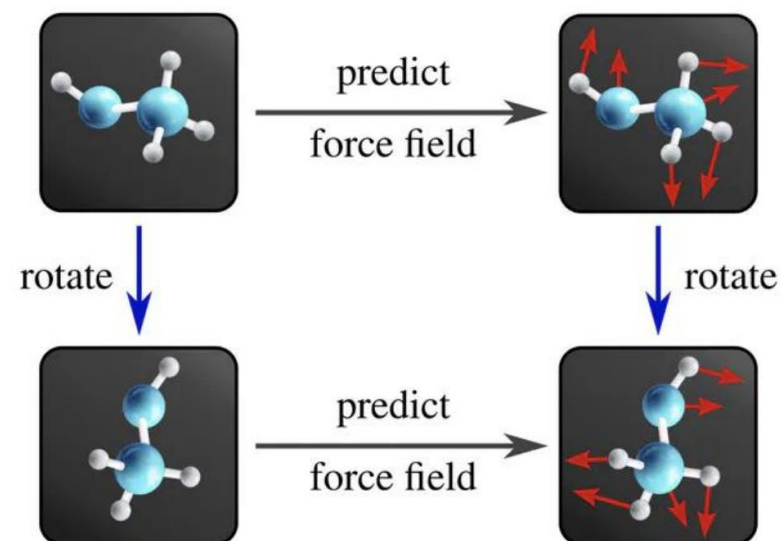
- e.g. the energy of a molecule, the total heat flux



- f** is **equivariant** if transforming the input transforms the output in the same way:

$$f(g \cdot x) = g \cdot f(x)$$

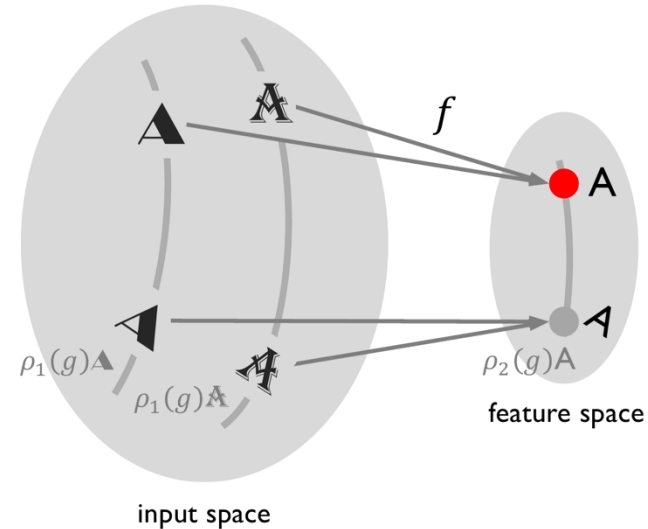
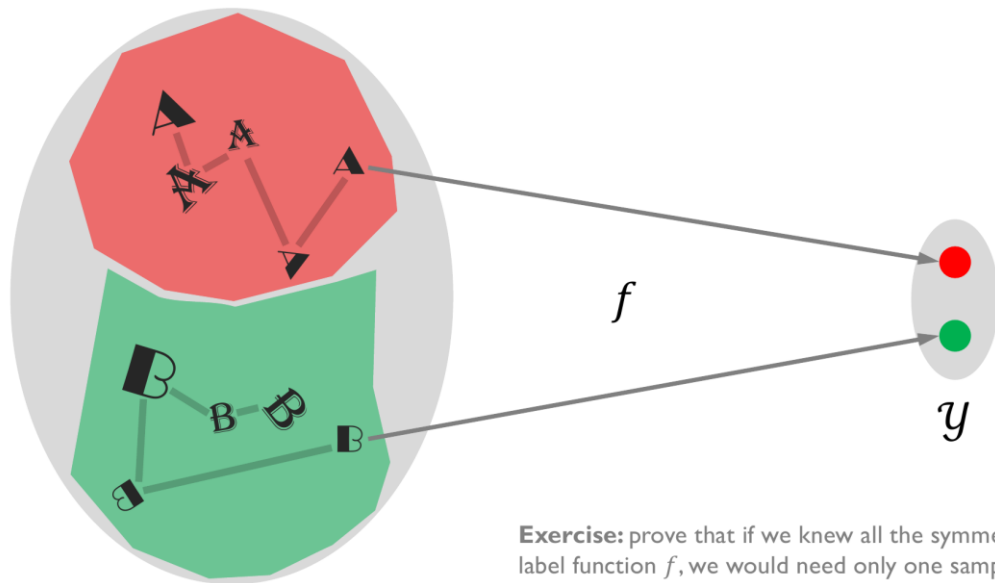
- e.g. the forces on its atoms, the next frame of a field



Next-frame prediction for the heat equation should be equivariant to D4. SmallCNN is only approximately so.

Symmetry and data efficiency

- The **orbit** of an input is the set of all its transformed copies. If the label is invariant, every point on an orbit has the same label
- If we knew all the symmetries of the label, one sample per orbit would be enough
- For an equivariant model, the output for one input fixes the output on its whole orbit. The model generalizes consistently to transformed inputs

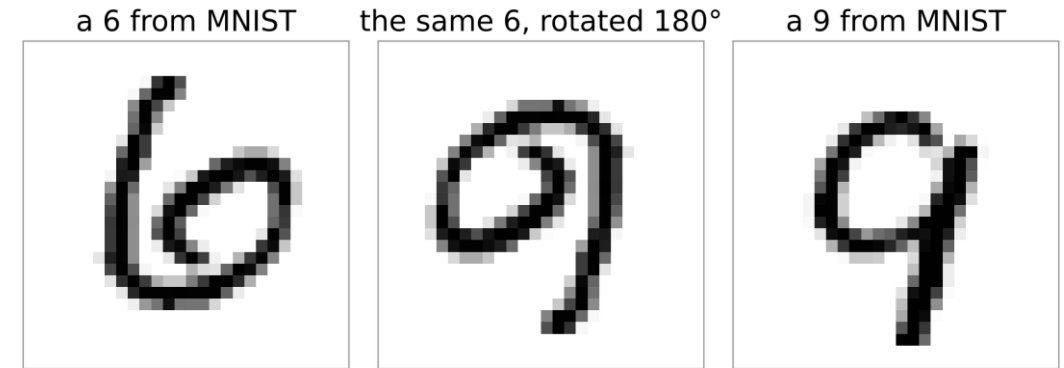


$$f(\rho_1(g)\mathbf{A}) = \rho_2(g) f(\mathbf{A})$$

$$f(\rho_1(g)\mathbf{A}) = \rho_2(g) f(\mathbf{A})$$

Choosing the symmetry group

- **MNIST**: a 6 rotated by 180° looks like a 9. A classifier invariant to all rotations cannot tell them apart. Rotations by a few degrees are safe
- **Heat equation** on the square: equivariance to all 8 transformations in D_4
- **BubbleML**: left-right reflection only
- Choose the group from the physics of the problem. Building in a symmetry the problem lacks is a modeling error

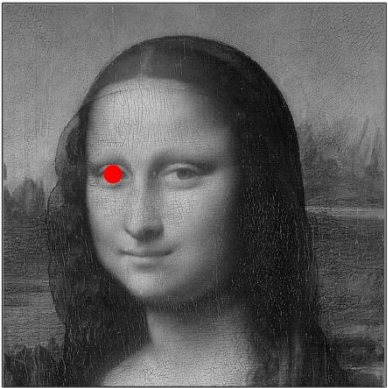


How scalar and vector fields rotate

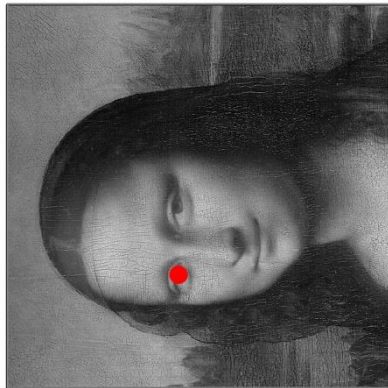
- **Scalar fields** (temperature, pressure, dfun) have one number per cell. Rotating the field moves the values
- **Vector fields** (velocity) also have a direction at every cell. Rotating the field moves the vectors and rotates each one. Rotating v_x and v_y as two independent images gives a wrong field, a common bug in **data augmentation**.

$$T'(\mathbf{x}) = T(R^{-1}\mathbf{x})$$

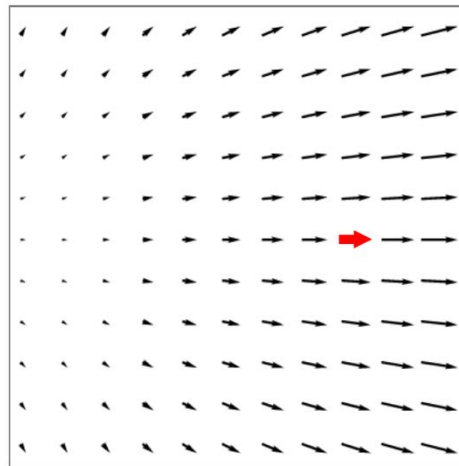
$$\mathbf{v}'(\mathbf{x}) = R \mathbf{v}(R^{-1}\mathbf{x}), \quad R = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} : (v_x, v_y) \rightarrow (-v_y, v_x)$$



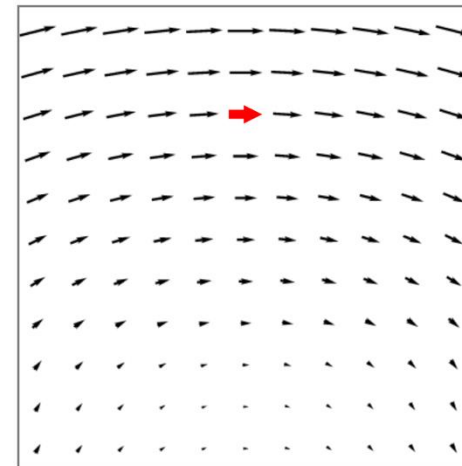
$T(\mathbf{x})$



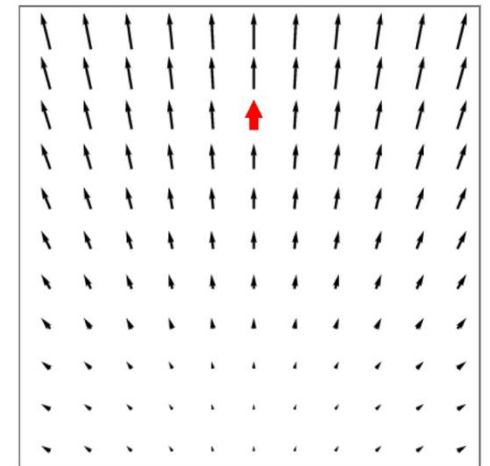
$T(R^{-1}\mathbf{x})$



$\mathbf{v}(\mathbf{x})$



$\mathbf{v}(R^{-1}\mathbf{x})$



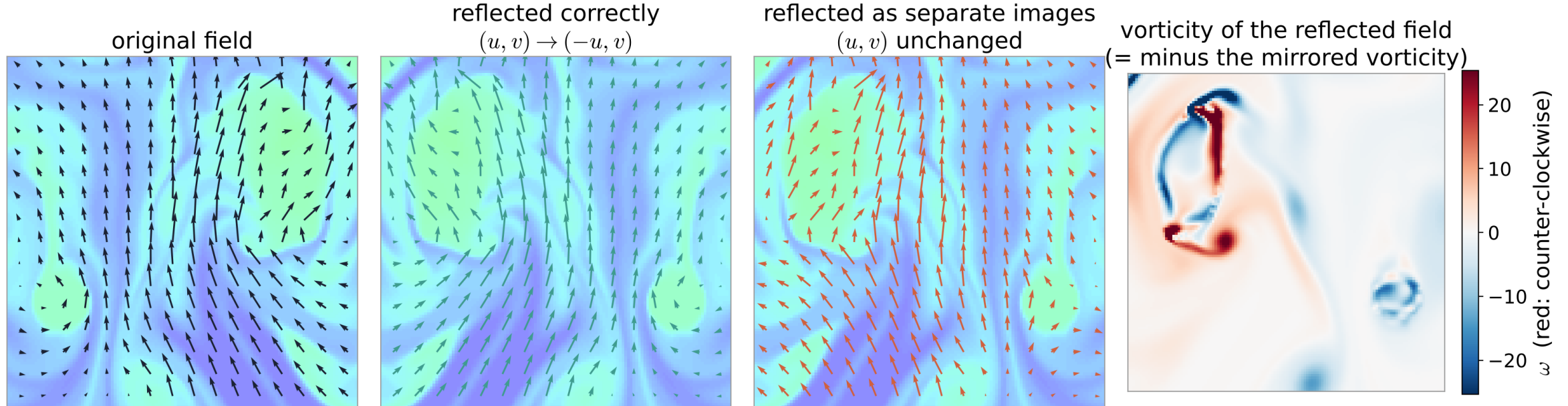
$R \mathbf{v}(R^{-1}\mathbf{x})$

A reflection flips one velocity component

- A left-right mirror maps x to $-x$. The horizontal velocity changes sign; the vertical velocity keeps its sign:

$$(u, v) \rightarrow (-u, v), \quad \omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \rightarrow -\omega$$

- Vorticity** ω measures local spinning. A mirror turns counter-clockwise spinning into clockwise, so ω changes sign
- For BubbleML augmentation: mirror every field, then **negate u**



Building symmetry into a model

	How	Holds	Cost
Data augmentation	train on transformed copies of the data	approximately, on inputs similar to the training data	more training; none at inference
Group averaging	average the outputs over all group elements	exactly , for any network	$ G $ forward passes
Equivariant layers	restrict the weights so every layer is equivariant	exactly , for all inputs and weights	a new layer design for each group

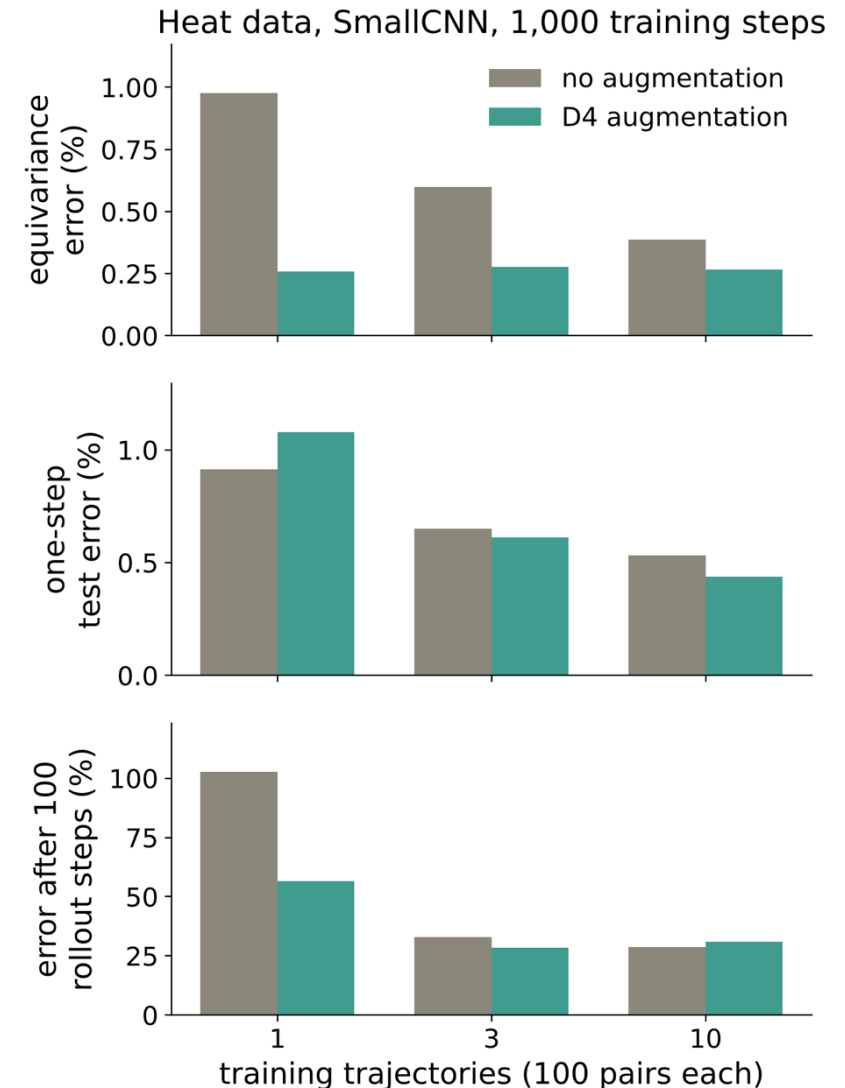
- For point clouds and graphs, sum or average over neighbors outputs the same result for every ordering (Deep Sets; PointNet)
- A fourth option adds a penalty for equivariance error to the loss: a soft constraint

Data augmentation

- For each minibatch, pick a random **g** from the group and transform both the input and the target. The loss averages over the group:

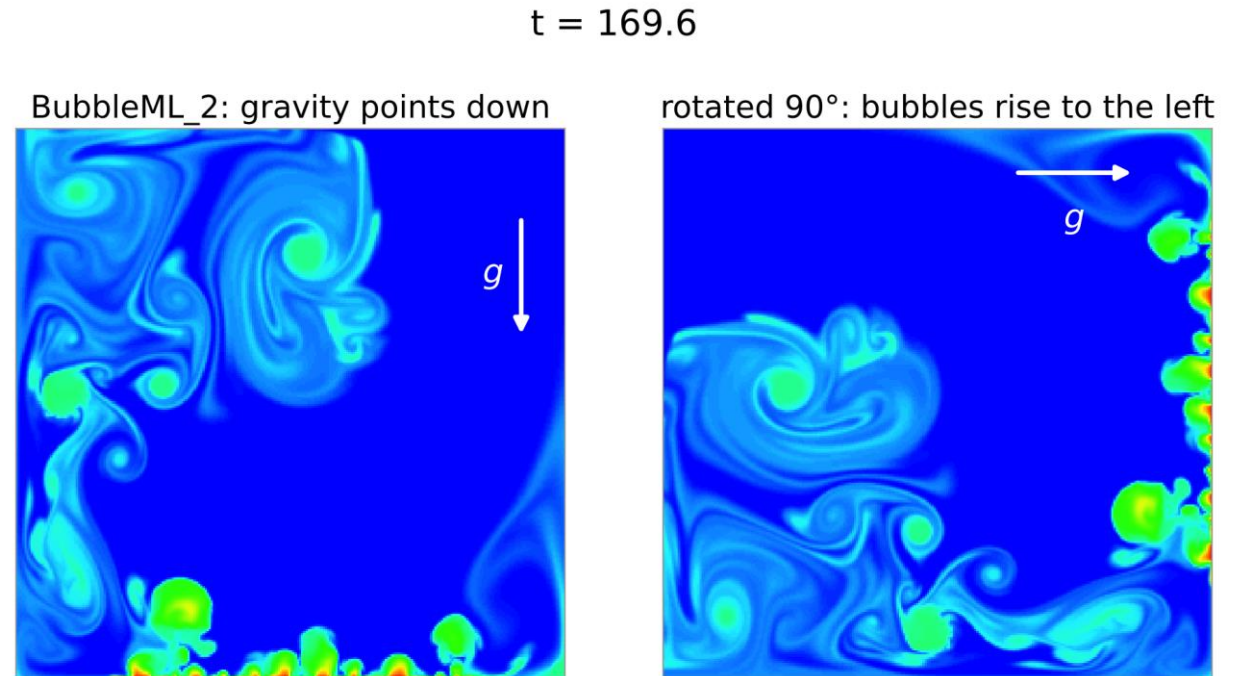
$$\mathcal{L}(\theta) = \mathbb{E}_{(x,y)} \mathbb{E}_g \|f(g \cdot x; \theta) - g \cdot y\|^2$$

- Transform each field by its type
- The model sees the symmetry only through data, so it learns it approximately. It also spends capacity on learning it, for example by learning rotated copies of the same filter
- Augmentation lowers the equivariance error at every training data size, but never to zero
- It helps most when data are scarce, especially in long rollouts.
- With a fixed training budget, it can **cost some one-step accuracy**, since the same steps now cover 8x as much data



Augmenting with the wrong symmetry teaches the wrong physics

- Rotating a BubbleML frame by 90° gives a valid array that describes boiling with gravity pointing sideways
- Its target is the rotated next frame, with bubbles rising to the left. The training set now contains two different physical problems
- The input carries no information about gravity, so the model cannot tell them apart
- Use only symmetries of the full problem. For BubbleML: left-right reflection

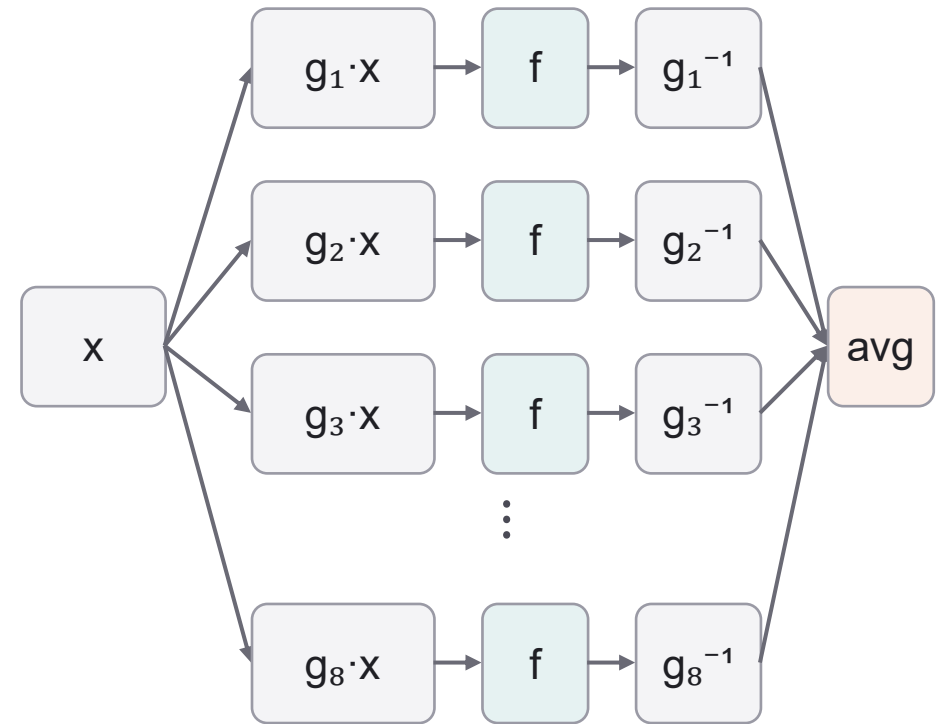


Group averaging

- Transform the input, apply the model, undo the transformation, and average over the group:

$$f_G(x) = \frac{1}{|G|} \sum_{g \in G} g^{-1} \cdot f(g \cdot x)$$

- f_G is equivariant for any f : transforming the input only reorders the terms of the sum
- Applied to a trained SmallCNN: the equivariance error drops to **round-off**, and the test error drops too, since averaging cancels errors that depend on direction. No retraining
- Costs $|G|$ forward passes, 8 for D4. For rotations by any angle the sum becomes an integral and has to be approximated
- A cheaper variant rotates each input to a standard orientation, predicts, and rotates back (**canonicalization**)



same weights in all 8 branches

Convolution commutes with shifts

- Convolution applies the same weights **w** at every position. Shifting the input by **s** cells shifts the output by **s** cells:

$$(w * u)_{i,j} = \sum_{a,b} w_{a,b} u_{i+a, j+b}, \quad w * (Su) = S(w * u)$$

- Example: $w = (1, -2, 1)$, the 1D second-difference stencil, on a row of 8 cells with zero padding

input u

0	0	1	3	1	0	0	0
---	---	---	---	---	---	---	---

output $w * u$

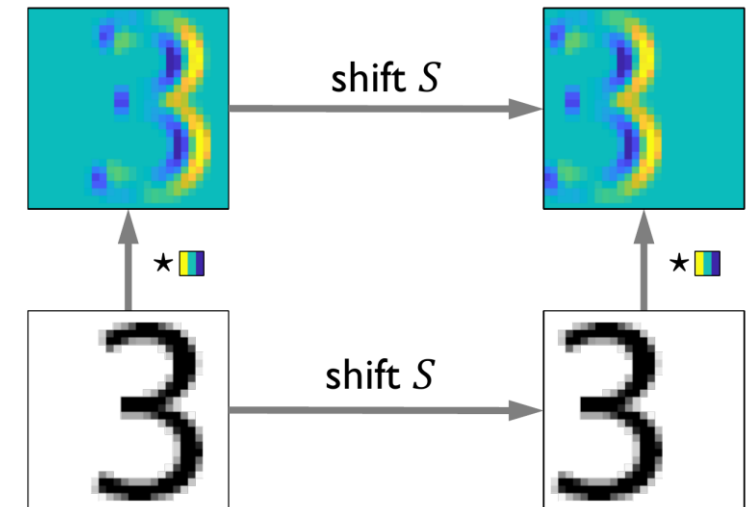
0	1	1	-4	1	1	0	0
---	---	---	----	---	---	---	---

input shifted by 2: Su

0	0	0	0	1	3	1	0
---	---	---	---	---	---	---	---

output $w * (Su)$

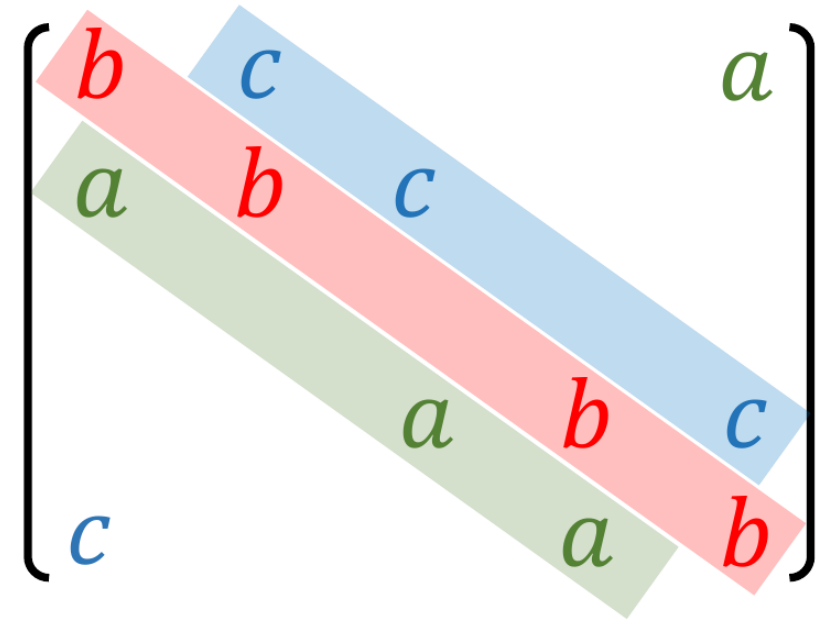
0	0	0	1	1	-4	1	1
---	---	---	---	---	----	---	---



Weight sharing makes every convolution layer shift-equivariant.

Every linear shift-equivariant layer is a convolution

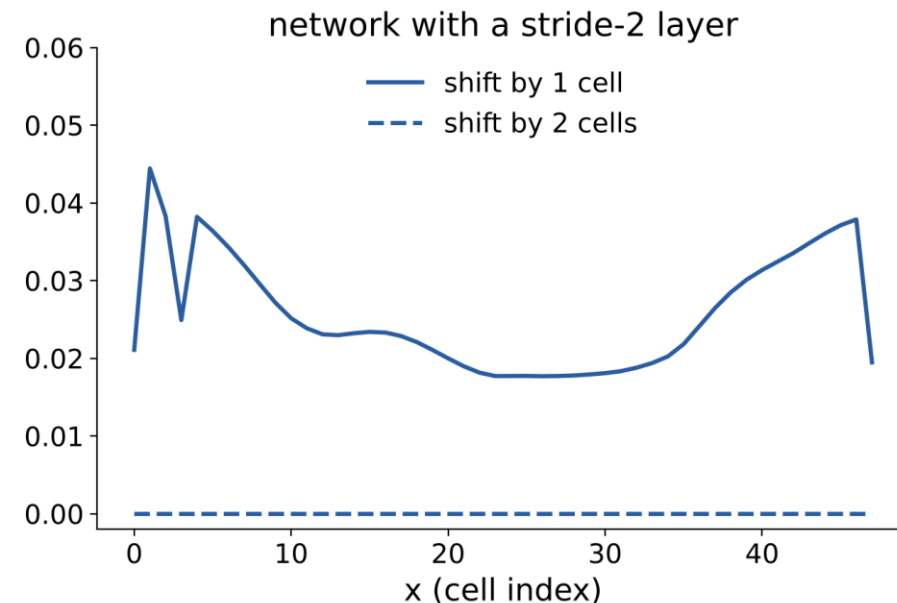
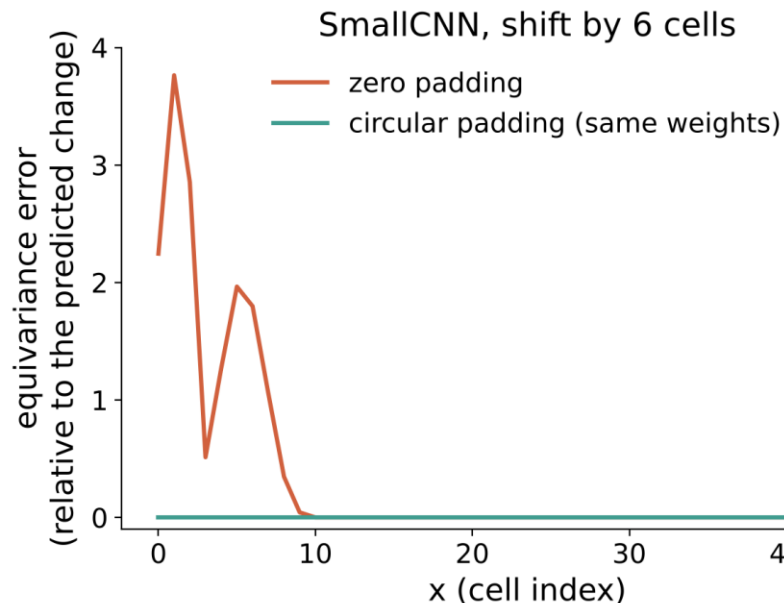
- Write a shift by one cell as a matrix $\mathbf{S}$. A linear layer $\mathbf{y} = \mathbf{C}\mathbf{x}$ is shift-equivariant if $\mathbf{C}\mathbf{S} = \mathbf{S}\mathbf{C}$
- This condition forces every row of $\mathbf{C}$ to hold the same weights, shifted by one column: a **circulant matrix** (right, with weights a , b , c)
- Multiplying by a circulant matrix is a convolution with (a, b, c) . Convolution is the only linear layer that commutes with shifts
- The same argument with rotations instead of shifts gives group convolutions



Convolution follows from translation symmetry.

What breaks shift equivariance in a CNN

- **Padding**: near a wall, the stencil reads padded values (Lecture 4). With zero padding, the error stays within 4 cells of a wall, the reach of SmallCNN's four 3 x 3 layers
 - circular padding, same weights: exact, as on a periodic domain
- **Stride and pooling**: a stride-2 layer keeps every other cell, so a shift by one cell changes which cells are kept. This is aliasing. Shifts by 2 cells stay exact
- **Coordinate channels** break it on purpose, when position matters (Lecture 4)



A rotation-invariant kernel has three free weights

- Require a 3 x 3 kernel to be unchanged by all 8 elements of D4: the 4 edge weights must be equal, and the 4 corner weights must be equal
- 9 weights become 3: center m, edge e, corner c
- The Laplacian stencil from Lecture 3 has this form, with $m = -4$, $e = 1$, $c = 0$
- A CNN composed of such kernels is **D4-equivariant**
- Each kernel weighs all directions equally, so the network **loses the direction** of edges and flows

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

general kernel: 9 weights

c	e	c
e	m	e
c	e	c

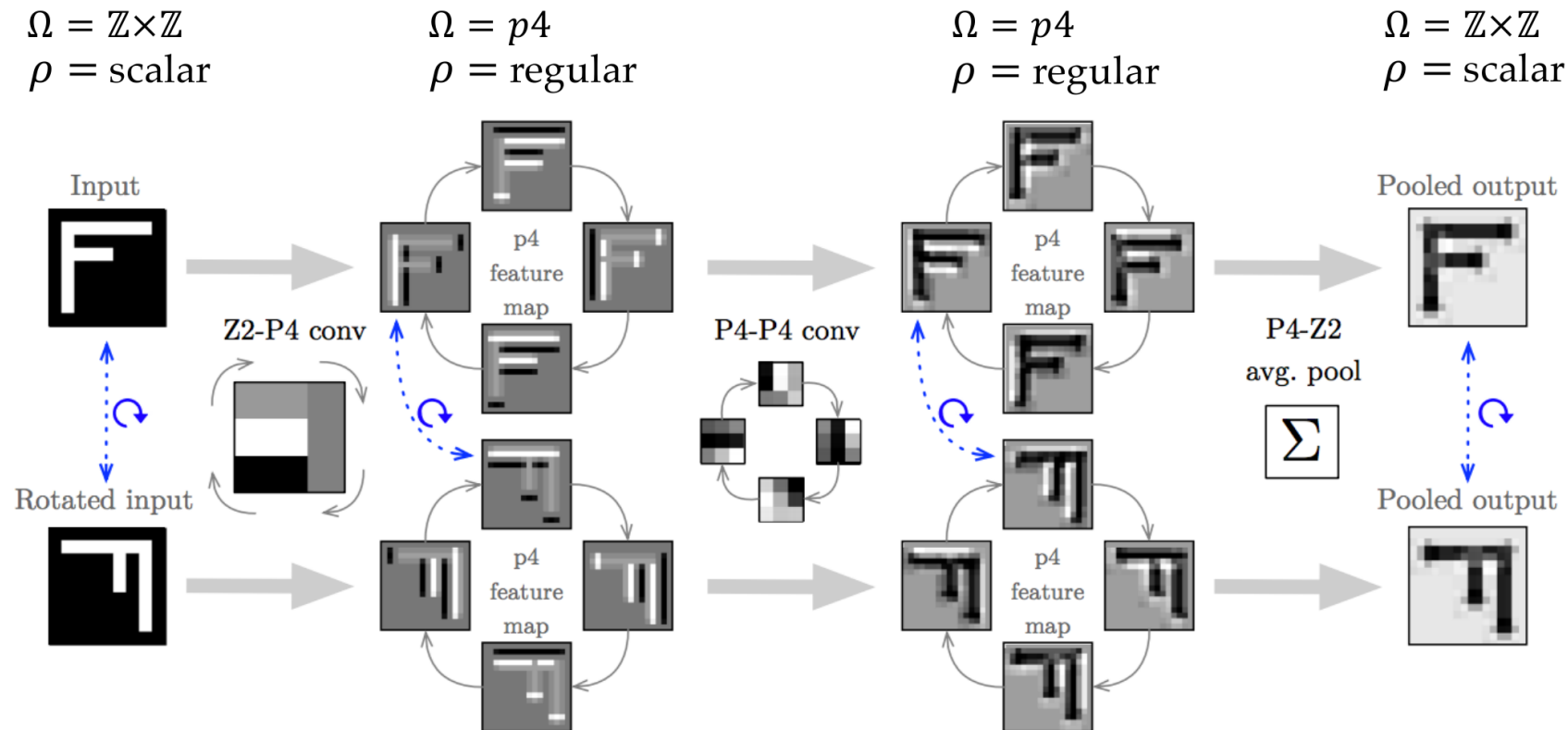
D4-invariant kernel: 3 weights

0	1	0
1	-4	1
0	1	0

Laplacian stencil (Lecture 3)

Group convolutions

- Apply one kernel in all 4 rotations to get 4 **orientation channels** (a lifting layer). Rotating the input rotates each channel and moves it to the next one, so direction information is kept
- Later layers share weights across rotations, so the network is D4-equivariant. Cost: 4 times the activations for the same number of weights

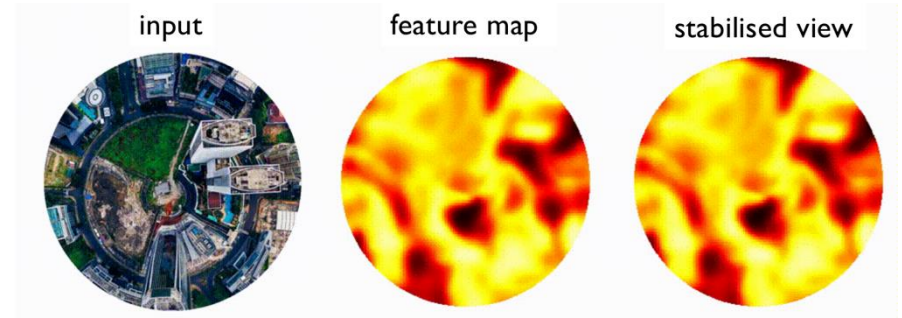


Veeling et al. 2018

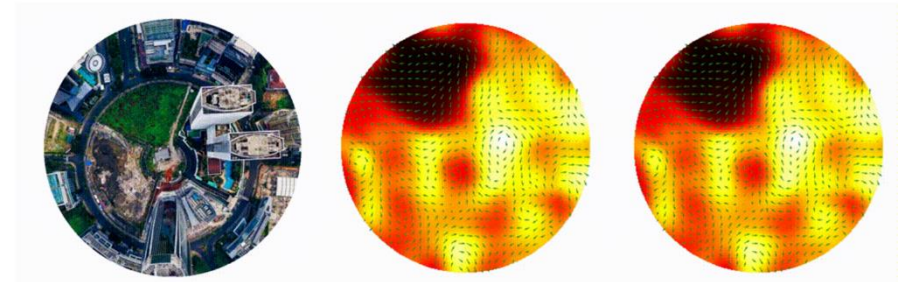
Symmetry in scientific ML

- **Turbulence and ocean currents:** CNNs equivariant to the symmetries of Navier-Stokes generalize better to transformed flows (Wang, Walters & Yu, ICLR 2021)
- **Neural PDE solvers:** augmenting with the symmetries of the PDE improves accuracy, most when data are scarce (Brandstetter, Welling & Worrall, ICML 2022)
- **Neural operators:** group-equivariant Fourier neural operators for rotations and reflections (Helwig et al., ICML 2023)
- **Interatomic potentials:** E(3)-equivariant graph networks match earlier models with up to **1000 times less** training data (NequIP, Batzner et al., Nature Communications 2022)
- **Pathology images:** rotation-equivariant CNNs without augmentation beat CNNs with augmentation, with a fraction of the data (Bekkers et al., MICCAI 2018; Lafarge et al., MedIA 2021)

CNN



Rotation-equivariant CNN



Weiler, Cesa 2019

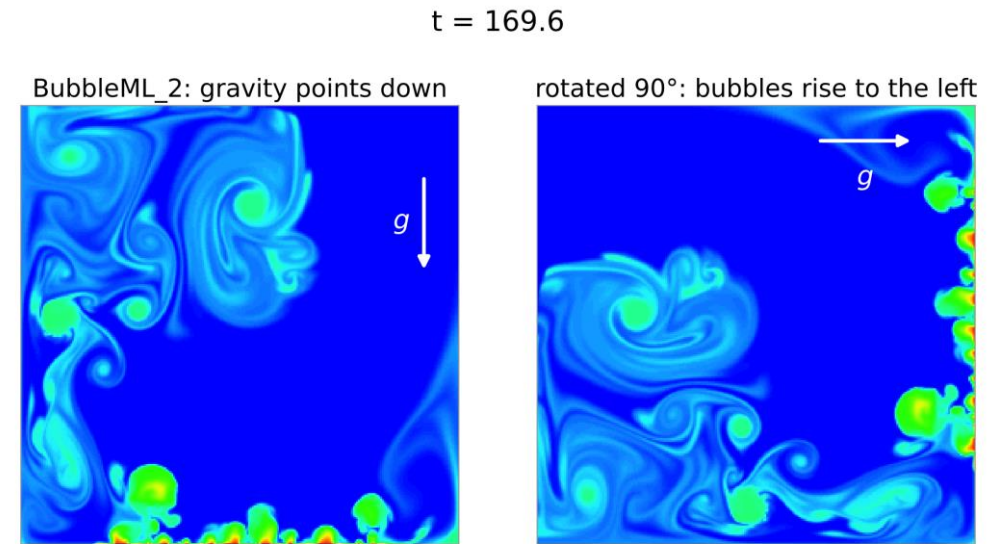
The common gains: data efficiency, and correct predictions for transformed inputs never seen in training.

Gravity as an input

- Gravity breaks rotation symmetry in BubbleML. Give the model the gravity direction $\mathbf{d}$ as an input, and rotate it along with the fields:

$$f(g \cdot x, R_g \mathbf{d}) = g \cdot f(x, \mathbf{d})$$

- Rotating the frame and gravity together is a symmetry of the physics, so a rotation-equivariant model is correct
- The same idea covers a heated wall, a mean flow direction, a magnetic field
- When the data are only nearly symmetric, **approximately equivariant** networks add a small learned term that breaks the symmetry where the data require it



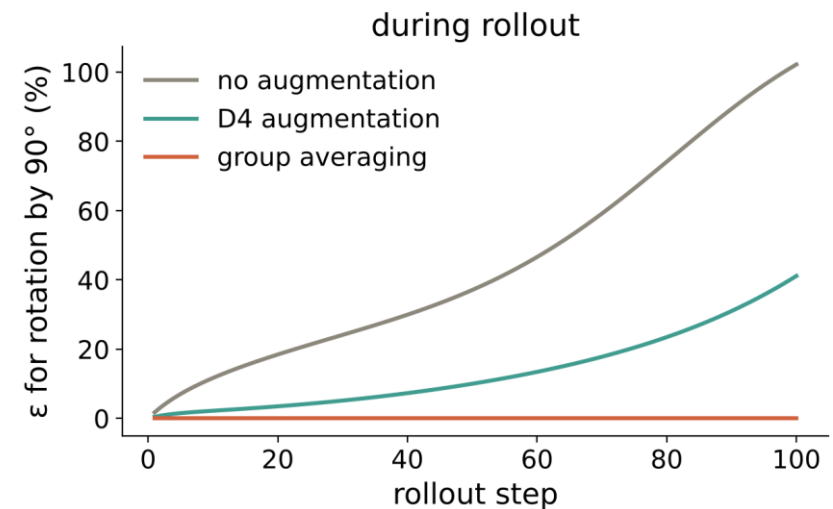
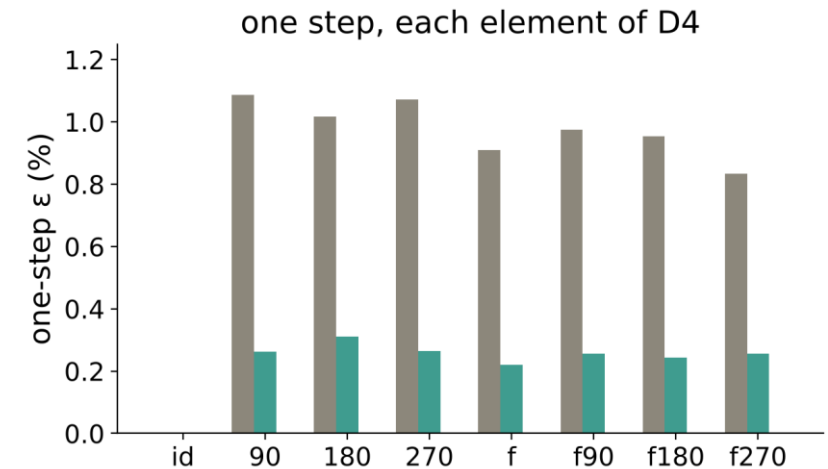
consistent once $\mathbf{d}$ rotates with the frame

Measure equivariance error

- Report the equivariance error on test inputs for every group element:

$$\varepsilon_g = \frac{\|f(g \cdot x) - g \cdot f(x)\|}{\|f(x)\|}$$

- One step: the errors are small and similar for every group element
- In rollout they **compound**: the error grows with every step. Augmentation slows the growth
- Group averaging stays at round-off for every element and every step
- An exactly equivariant model still needs this check: padding, stride and interpolation can break it



f: flip left-right; 90, 180, 270: rotation in degrees

Summary

- A symmetry maps solutions to solutions. It comes from the equation, the domain and the boundary conditions together
- Symmetries form groups: discrete (D4 on a grid) or continuous (rotations by any angle). Off-grid rotations need interpolation
- Invariant outputs stay the same; equivariant outputs transform with the input
- Scalars move; vectors move and rotate. A reflection flips one velocity component and the sign of the vorticity
- Augmentation is approximate, group averaging is exact at $|G|$ times the cost, equivariant layers are exact by design
- Convolutions commute with shifts away from walls; padding and stride break this. Group convolutions add rotations
- When a known field breaks a symmetry, make it an input. Always measure the equivariance error

After class

- **Bronstein, Bruna, Cohen & Veličković, *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges* (2021), geometricdeeplearning.com**
 - Ch. 3: foundations of equivariant deep learning · Ch. 6: grids · Ch. 7: group convolution · Ch. 8: geometric graphs (molecules)
- **Prince, *Understanding Deep Learning* (udlbook.github.io)**
 - Section 10.1: invariance and equivariance · Ch. 13: graph networks and permutations
- **Bekkers, *An Introduction to Group Equivariant Deep Learning*, lecture videos and notebooks (uvagedl.github.io)**
- **Cohen & Welling, *Group Equivariant Convolutional Networks*, ICML 2016**

Next, Lecture 6 (Tue, Oct 13): CNN and U-Net architectures for spatial fields

NEXT: HANDS-ON

From concepts to code

→ [Open the Lecture 5 notebook](#)

1. Measure the equivariance error of the Lecture 4 SmallCNN for all 8 elements of D4, and for shifts with zero and circular padding
2. Wrap the model in group averaging; check the error and the cost
3. Train with D4 augmentation on a small training set; compare test error, equivariance error and rollouts